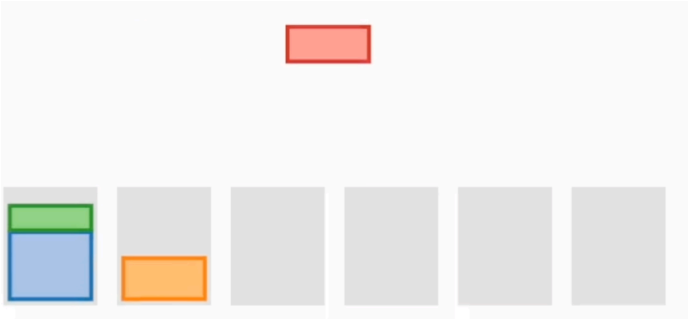
Project Topic: Automated Heuristic Design via Large Language Models

> **Before you start.** Every direction on this page uses a large language model (LLM). No LLM API credits are provided this year: arrange your own access (a commercial API reachable in Mainland China, or an open-weight model run locally) and state your plan in the proposal; see Section 3 of the Course Project page. Links marked † are hosted by Google or YouTube and may not be reachable from Mainland China; an alternative is given next to each.

Background

A heuristic is a function that estimates how close a candidate solution is to a goal; for example, the Euclidean (straight-line) distance between two cities is a heuristic for estimating how close a city is to the goal city. Traditionally, heuristics are designed manually by human experts via tedious trial and error. In this project, we would like to explore how the design of a heuristic can be formulated as a search problem and how we can leverage LLMs to search over the space of heuristics and find better heuristic functions for a given problem.

Specifically, let us consider the online bin packing problem we discussed in class as an example. To recap, the bin packing problem is an optimization problem in which items of different sizes must be packed into a finite number of bins or containers, each of a fixed given capacity, to minimize the number of bins used. In the online version, items of different volumes arrive sequentially, and the decision maker has to decide where to pack the currently observed item; each decision is made without recall. See the figure below for a pictorial illustration.



Online bin packing: the red item has just arrived and must be placed now.

A generic template of the heuristic for online bin packing problems can be coded in Python as follows:

```
def heuristic(bins, item):
    # input arguments:
    #    bins: the current capacities of each bin
    #    item: the size of the item to be packed
    # output:
    #    index of the bin for holding the item

    # ------------ start of heuristic function -------------- #
```

```
    index = ...  # to be determined
    # ------------ end of heuristic function -------------- #
    return index
```

Then, a simple heuristic that packs items according to the remaining capacities of the bins can be implemented as follows:

```
def heuristic(bins, item):
    # input arguments:
    #    bins: the current capacities of each bin
    #    item: the size of the item to be packed
    # output:
    #    index of the bin for holding the item

    # ------------ start of heuristic function -------------- #
    # the remaining capacity of each bin if the item is placed in
every bin
    remaining_capacity = bins - item
    # find the index of the bin with least capacity remaining
    index = argmin(remaining_capacity)
    # ------------ end of heuristic function -------------- #
    return index
```

Now, instead of relying on human intuition or manual trial and error to find a better heuristic, we would like to automate the design process by treating it as a search problem. One successful realization of this idea is presented in a recent Google DeepMind paper published in *Nature*: [Mathematical discoveries from program search with large language models](). The method is called FunSearch, and the main idea is illustrated in the following figure.

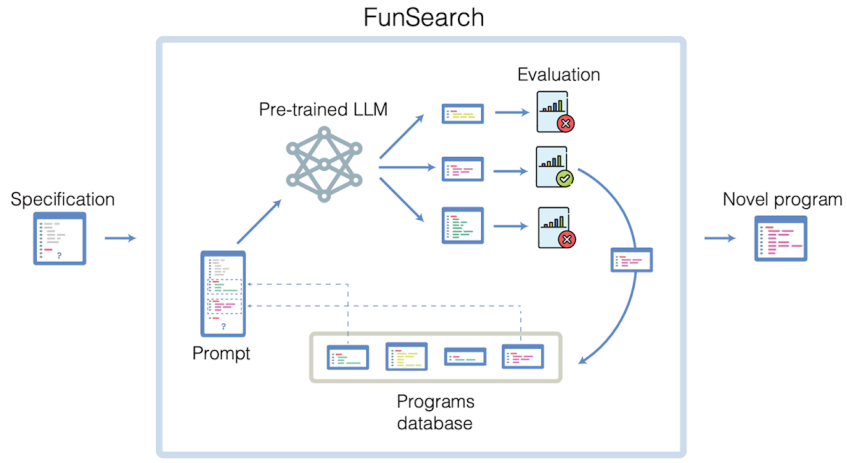


Figure credit: Romera-Paredes et al., *Nature* 625, 468–475 (2024).

FunSearch refers to a heuristic function as a program. FunSearch first asks an LLM to create a set of programs, each of which is incorporated into the general heuristic code template shown above. Each program is evaluated on some test instances of the online bin packing problem, and the resulting performance is recorded. Then the main loop of FunSearch starts: in each iteration, FunSearch picks a promising program (in terms of

performance) from the set of programs created before, sends it to the LLM for modification so that the performance can be improved, evaluates the modified program (again on test instances), and keeps it if its performance improves. This process repeats for many iterations until a novel program is obtained or a pre-defined maximum number of iterations is reached.

Useful resources

- FunSearch
  - Paper: nature.com/articles/s41586-023-06924-6 (open access)
  - Blog post†: deepmind.google/discover/blog/funsearch-making-new-discoveries-in-mathematical-sciences-using-large-language-models — the paper covers the same content.
  - Official implementation by Google DeepMind: github.com/google-deepmind/funsearch (note that this implementation does **not** include the code for (i) the LLM interface and (ii) the performance evaluation of a program, referred to as the "sandbox" in the code).
  - A Jupyter notebook comparing the heuristics designed by human experts and by FunSearch: bin_packing/bin_packing.ipynb in the same repository.
  - A functional implementation of FunSearch on online bin packing, provided for you: github.com/RayZhhh/funsearch (run it locally; the Google Colab version is not available from Mainland China).
- Online bin packing datasets: OR-Library; Weibull instances are generated as in the FunSearch paper and repository.
- Other related works: AEL (arXiv:2311.15249), GLS (arXiv:2401.02051), AlphaGeometry (*Nature* 625, 476–482, 2024).

Potential project directions

Direction 1. Application to other search problems beyond packing

Note that it is not enough to simply apply FunSearch to a new problem. We expect to see some form of novelty (e.g. a modified version of FunSearch for your specific problem).

- **Traveling salesman problem (TSP)**
  - Overview†: developers.google.com/optimization/routing/tsp — alternative: the OR-Tools repository github.com/google/or-tools (documentation and examples included) or any textbook treatment.
  - Benchmarks: TSPLIB; symmetric instances (recommended): TSPLIB95/tsp.
  - Existing algorithms: constructive heuristics (nearest, cheapest, farthest insertion) and a Gurobi model in attention-learn-to-route; the LKH-3 solver (C source, Python wrapper); neural solvers AM and POMO; Google OR-Tools.
- **Flow shop scheduling**
  - Overview†: developers.google.com/optimization/scheduling/job_shop (flow shop scheduling is a special case of job shop scheduling in which all jobs have the same execution order) — alternative: the OR-Tools repository above.
  - Benchmarks: OR-Library; Taillard instances; VRF instances.

    - Existing algorithms: Google OR-Tools; simple heuristics in flowshop-scheduling.
- **Capacitated vehicle routing problem (CVRP)**
    - Overview†: developers.google.com/optimization/routing/cvrp — alternative: the OR-Tools repository above.
    - Benchmarks: CVRPLIB (Uchoa et al. and the A, B, E, F, M, P sets, recommended); SINTEF TOP for CVRPTW and PDPTW.
    - Existing algorithms: 15 classical heuristics in VeRyPy; Google OR-Tools; LKH-3 (links above); HGS-CVRP; neural solvers AM and POMO (links above); AILS-CVRP for large instances.
- **More problems**: the OR-Tools examples† at developers.google.com/optimization/introduction/python, mirrored in the examples folder of the OR-Tools repository.

Direction 2. FunSearch enhancements

FunSearch provides a simple and generic implementation of the idea of using an LLM to automate heuristic design as program search. Under this topic we seek enhancements that make FunSearch more effective, efficient, and robust. Some ideas to get you started (it is fine to use the online bin packing problem for this topic; feel free to use other problems):

- **Thoughts-augmented FunSearch.** In addition to code, can we also use thoughts or ideas, expressed as natural-language descriptions, to enhance FunSearch? For example, supply a short description of what the heuristic should do together with its code to the LLM, and see whether the LLM generates better heuristics faster.
- **Sample-efficient FunSearch.** Many programs are created as FunSearch progresses, and the chance that a newly sampled program is functionally similar to an existing one increases. Instead of blindly evaluating every program the LLM creates, can we design a duplicate-checking mechanism that avoids evaluating a program that has already been evaluated? The check should be quicker than the evaluation itself. Similarity should be defined at the level of functionality: the two programs below look different but compute the same thing.

```
def mean(xs):
    return sum(xs) / len(xs)
```

```
def mean(xs):
    total, n = 0, 0
    for x in xs:
        total += x
        n += 1
    return total / n
```

- **Novelty-driven FunSearch.** Instead of always finding the best solution in terms of one performance metric, we often want to explore different ways of solving the same problem. Here this means finding programs with diverse behaviours (in terms of functional logic) that all fulfil the goal reasonably well. Take a look at Kenneth Stanley's talk “Why Greatness Cannot Be Planned” (also a book of the same title) to

get ideas; essentially, replace the search routine in FunSearch with the novelty search algorithm described there.

- **Multi-objective FunSearch.** Besides performance, one may desire other properties of a heuristic, for example interpretability (how easily can humans comprehend it?) and simplicity. Can we add such criteria to FunSearch and find heuristics that satisfy several of them at once?
- **Heuristic-oriented search space and distance study.** Add a distance function that measures the difference between heuristics (see the work on distances in evolutionary computation and on code embeddings in machine learning), and use it for advanced population management: multiple populations, maintaining diversity, allowing infeasible heuristics.

Direction 3. Adaptation of FunSearch to machine learning applications

Instead of using LLMs to evolve programs as heuristic functions that guide a search algorithm, can you adapt the same idea to develop programs for machine learning tasks?

- **Neural architecture search.** A deep neural network can be expressed as Python code using PyTorch or TensorFlow (an example is below), so FunSearch can be applied to design a network as a search problem. Look for NAS benchmarks (for example the NAS-Bench family) so that candidate architectures can be evaluated cheaply.

```
import torch.nn as nn

model = nn.Sequential(
    nn.Conv2d(3, 32, kernel_size=3, padding=1), nn.ReLU(),
nn.MaxPool2d(2),
    nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(),
nn.MaxPool2d(2),
    nn.Flatten(), nn.Linear(64 * 8 * 8, 10),
)
```

- **Robotics control.** The control logic of a robot can also be expressed as a program; FunSearch can then search for a program that controls the robot to perform a task. Resources: arXiv:2307.16890; a related talk on YouTube† (youtube.com/watch?v=sEFP1Hay4nE) — the paper is sufficient.